

Iteration 3

Riley Ashok and Presthika Vijaykumar

March 2026

1 Basic Functionalities

Below are the basic functionalities for our drone fleet simulator, and what it must be able to do for the simulation to run:

Functionality	Description	Input	Output
Drone Registration	Adds a new drone to the fleet and initializes it in the system.	Drone ID Initial Position	Confirmation message; enables tracking and management of the drone
Mission Assignment	assigns a mission/shape formation to drones.	Drone ID(s) Mission type Target positions	Mission confirmation, allows coordinated drone tasks.
Path Planning	Calculates the optimal path for a drone.	Drone position Target position Obstacles	Waypoints/trajectory, ensures efficient and collision-free movement.
Collision Detection / Avoidance	Checks for potential collisions and adjusts paths.	Drone positions Velocity Proximity threshold	Adjusted paths/warnings/alerts, prevents collisions and ensures safety.
Simulation Update	Updates drone positions and mission progress.	Time step Drone commands	Updated positions and statuses, Advances simulation loop.
Visualization	Shows drone positions graphically, simulation data, 2D/3D, helps users understand drone behavior and mission progress.	Drone Positions Mission Plan	Graphical representation of drones, paths, and environment.

Table 1: Core functionalities of the drone fleet management system

2 Object Oriented Programming

Our drone fleet simulator will use four classes. One will be individual drones, one will control fleets, one will plan the mission for the fleets, and then the final one will develop simulations and graphics. A list of the classes as well as their attributes can be found below:

Class Name	Attribute / Method	Access Data Type	Description
Drone	droneID	private int	Unique ID number for drone
	position	private struct	current position of drone
	velocity	private float	current velocity of drone
	updatePosition()	public void	updates the position of drone
	detectCollision(otherDrone)	public bool	will tell if drone will collide with another drone
	addPosition()	public void	Add a new position for the drone
	getPosition()	public void	get the current position of the drone
	getPattern()	public void	get the pattern the drone is doing
	getDroneID()	public int	get the ID number of drone
FleetManager	setColor	public void	set the color of the drone LED
	drone[]	private int	array of drones in fleet
	droneAngle	private double	angle drones are flying in
	droneDistance[]	private double	array of drone distances from fleet
	dronePosition[]	private double	position of drones from fleet
	addDrone(droneID)	public void	adds a drone to the fleet
	deleteDrone(droneID)	public void	deletes a drone from the fleet
	missionID	private int	Unique ID for mission element
	missionType	private string	information about mission
MissionPlanner	planPath(droneID, target)	public vector	tell a drone where to go
	assignMission(Mission)	public void	assign a group of drones to a mission
	getMissionPlan()	public void	get the current mission of a group of drones
	deleteMission(missionID)	public void	delete a mission from the queue
	addMission()	public void	add a mission to the queue
SimulationEngine	timeStep	private float	How long mission element will take
	runStep()	public void	runs the mission element
	displayStatus	public void	displays graphics and status of fleet

Table 2: Objects for Drone Simulation Engine

3 System Overview

In this section, we will discuss the inputs, process, and output for the drone fleet simulator.

3.1 Input

1. Drone data:
 - droneID
 - position
 - velocity
2. Mission assignments:
 - shape
 - target positions
3. Simulation parameters
 - time step
 - environment data

3.2 Process

1. User can create drones and fleets, setting beginning parameters
2. User enters shapes for drone show
3. MissionPlanner calculates optimal paths
4. FleetManager updates drone positions
5. Collision detection checks for other drones and potential crashes
6. Collision avoidance adjusts drone paths
7. Simulation Engine updates all statuses and generates graphics

3.3 Output

- Updated drone positions
- Collision avoidance actions
- Mission progress updates
- Visualization of drone show
- Errors if drones will collide or are in the same location
- Error if shape or positions are impossible for drone

A detailed diagram of the system overview can be seen below:

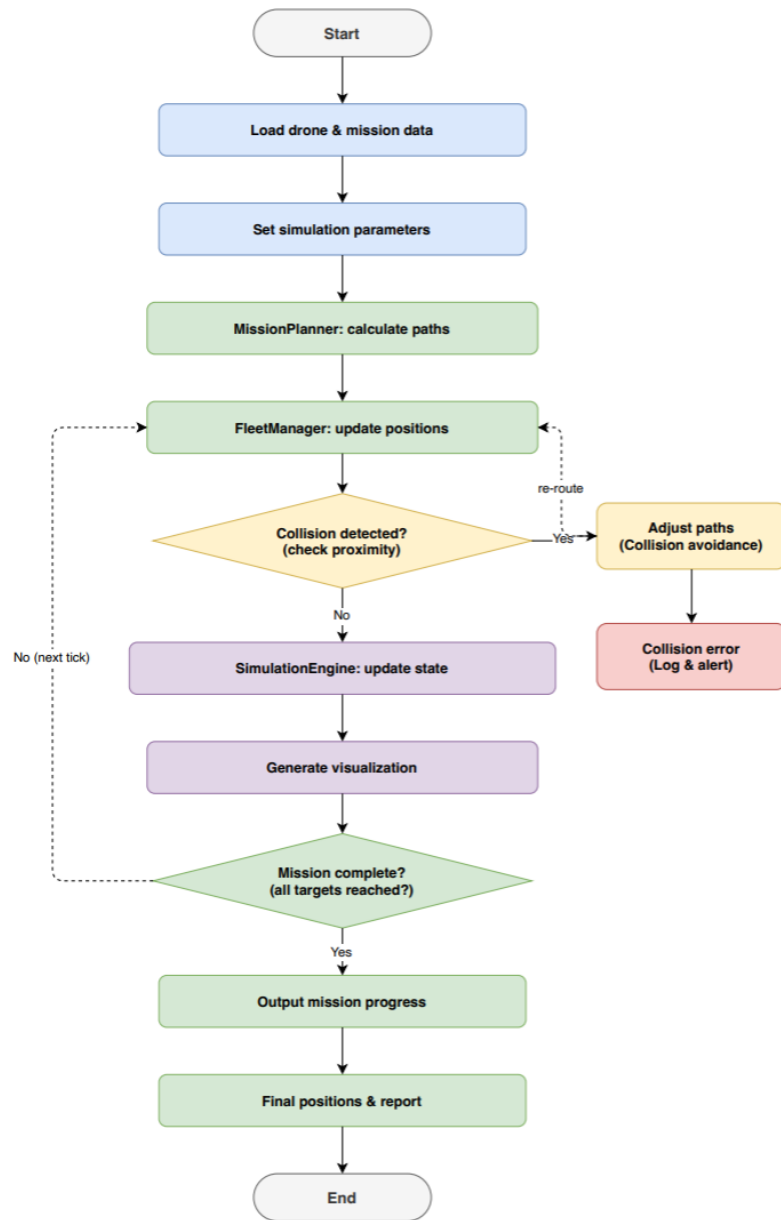


Figure 1: System Process Flowchart